

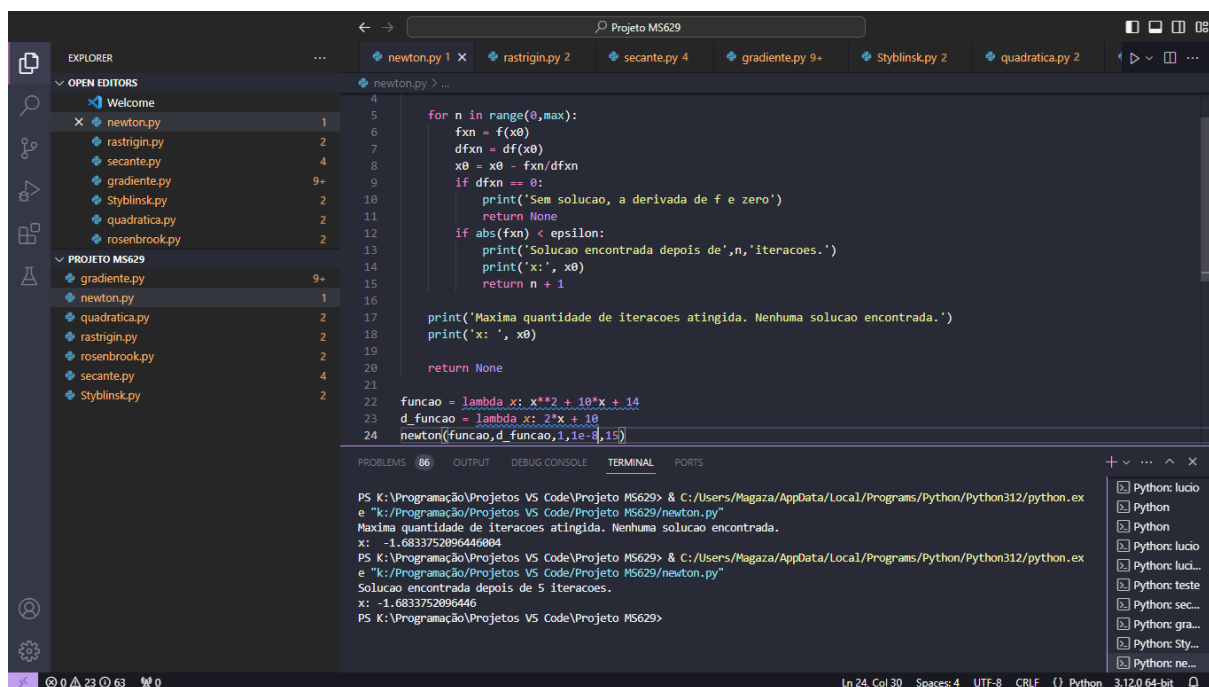
MS629 2S/2023 - Lúcio Tunes

Projeto Computacional

Luís Roberto Marques Junior 173216
João Vitor de Oliveira Perri 204515
João Vitor souza de Alcantara 215351

O objetivo desse projeto é a implementação de três algoritmos distintos de forma computacional, sendo eles o método de Newton, o método do Gradiente e o método das Secantes. Após as implementações deve-se comparar o desempenho deles com relação ao número de dimensões (quantidade de variáveis), com relação a precisão ϵ e com relação a escolha dos pontos iniciais x^0 (x^0 e x^1 , no caso da secante). Essa comparação será feita com base em quatro funções pré estabelecidas: uma função quadrática, a função de Rosenbrook, a função Styblinsky-Tang e a função Rastrigin.

Os algoritmos foram implementados em Python. Com todos os algoritmos implementados (Figuras 1-3), foi possível a execução e comparação deles através das funções fornecidas.



```
4
5
6 for n in range(0,max):
7     fxn = f(x0)
8     dfxn = df(x0)
9     x0 = x0 - fxn/dfxn
10    if dfxn == 0:
11        print('Sem solucao, a derivada de f e zero')
12        return None
13    if abs(fxn) < epsilon:
14        print('Solucao encontrada depois de',n,'iteracoes.')
15        print('x:', x0)
16        return n + 1
17
18 print('Maxima quantidade de iteracoes atingida. Nenhuma solucao encontrada.')
19 print('x:', x0)
20
21 return None
22
23 funcao = lambda x: x**2 + 10*x + 14
24 d_funcao = lambda x: 2*x + 10
25 newton(funcao,d_funcao,1,1e-8,15)
```

PROBLEMS (86) OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS K:\Programação\Projetos VS Code\Projeto MS629> & C:/Users/Magaza/AppData/Local/Programs/Python/Python312/python.exe
e "k:/Programação/Projetos VS Code/Projeto MS629/newton.py"
Maxima quantidade de iteracoes atingida. Nenhuma solucao encontrada.
x: -1.6833752096446004
PS K:\Programação\Projetos VS Code\Projeto MS629> & C:/Users/Magaza/AppData/Local/Programs/Python/Python312/python.exe
e "k:/Programação/Projetos VS Code/Projeto MS629/newton.py"
Solucao encontrada depois de 5 iteracoes.
x: -1.6833752096446
PS K:\Programação\Projetos VS Code\Projeto MS629>
```

Figura 1. Execução e código do método de Newton para uma $f(x) = x^2 + 10x + 14$ de uma variável, com erro epsilon $1e-7$ e limite de 15 iterações.

```

14 x0 = np.array([10, 12, 3, 5, -15, 14]) #np.zeros(n)
15
16 def gradient(f, grad_f, alpha, beta, gamma, sigma, rho, epsilon, max_iter, x0):
17     k = 0
18     xk = x0
19
20     while k < max_iter:
21         grad_fk = grad_f(xk)
22         dk = -grad_fk
23
24         tk = 1.0
25
26         while f(xk + tk * dk) > f(xk) + alpha * tk * np.dot(grad_fk, dk):
27             tk *= sigma
28
29         xk1 = xk + tk * dk
30
31         if np.linalg.norm(grad_f(xk1)) < epsilon:
32             break
33
34     xk = xk1

```

Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Experimente a nova plataforma cruzada PowerShell <https://aka.ms/pscore6>

PS K:\Programação\Projetos VS Code\Projeto MS629> & 'C:\Users\Magaza\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\Magaza\vscode\extensions\ms-python.python-2023.28.0\pythonFiles\1lib\python\debugpy\adapter\...\.debugpy\launcher' '58429' '-.*' 'k:\Programação\Projetos VS Code\Projeto MS629\gradiente.py'

X0: [10 12 3 5 -15 14]
Resultado: [2.32791323e-07 0.00000000e+00 1.55096365e-25 0.00000000e+00
4.57914262e-20 -1.04308128e-07]
Número de iterações: 47
PS K:\Programação\Projetos VS Code\Projeto MS629>

Figura 2. Execução e código do método do Gradiente com $x^0 = [10, 12, 3, 5, -15, 14]$, com erro epsilon $1e-6$ com 47 iterações necessárias.

```

5
6 # metodo secante
7 f = lambda x, y: x**10 - 5*x**9 - 9*x**5 + 3*x**2 + x + y**5 - y**3
8
9 def secante(x0, y0, x1, y1, epsilon, max):
10     passo = 1
11     condicao = True
12     while condicao:
13         fx0y0 = f(x0, y0)
14         fx1y1 = f(x1, y1)
15         if fx0y0 == fx1y1:
16             print('Erro, divisão por zero')
17             break
18
19         x2 = x0 - (x1 - x0) * fx0y0 / (fx1y1 - fx0y0)
20         y2 = y0 - (y1 - y0) * fx0y0 / (fx1y1 - fx0y0)
21
22         print('Iteracao %d, x2 = %.6f, y2 = %.6f e f(x2, y2) = %.6f' % (passo, x2, y2, f(x2, y2)))
23         x0, y0 = x1, y1
24         x1, y1 = x2, y2
25         passo = passo + 1

```

Iteracao 1, x2 = 4.786590, y2 = 9.426820 e f(x2, y2) = -230416.833245
Iteracao 2, x2 = 4.975216, y2 = 9.049568 e f(x2, y2) = -13692.761617
Iteracao 3, x2 = 4.987133, y2 = 9.025733 e f(x2, y2) = 6923.589891
Iteracao 4, x2 = 4.983131, y2 = 9.033737 e f(x2, y2) = -107.175511
Iteracao 5, x2 = 4.983192, y2 = 9.033615 e f(x2, y2) = -0.818780
Iteracao 6, x2 = 4.983193, y2 = 9.033615 e f(x2, y2) = 0.000000
Iteracao 7, x2 = 4.983193, y2 = 9.033615 e f(x2, y2) = 0.000000

A raiz é: x = 4.98319275, y = 9.03361451
PS K:\Programação\Projetos VS Code\Projeto MS629>

Figura 3. Execução e código do método da Secante com x^0 e $x^1 = [4, 11], [5, 9]$ com erro epsilon $1e-8$ com 7 iterações necessárias e um limite de 30.

Todos os algoritmos em questão tem dificuldades com precisões muito elevadas (Figura 6), valores a partir da 8ª casa decimal ($1e-8+$), e não foi possível melhorar isso de forma significativa. Em alguns casos mais simples, como polinômios de dimensões menores, precisões mais elevadas ($1e-12$) puderam ser obtidas, mas nada consistente. Tentar calcular valores na precisão da máquina ($1e-16$), pelo menos com poucas iterações, não foi possível. O custo computacional sobe de forma sensível comparativamente a elevação da precisão pedida. De forma geral e para precisões razoáveis, o método de Newton é o mais

rápido (Figuras 4), com um comportamento razoável do método das secantes dadas boas condições iniciais; o Gradiente acaba por ser o pior nesse quesito, e dado seu passo lento, o custo escala bem rápido.

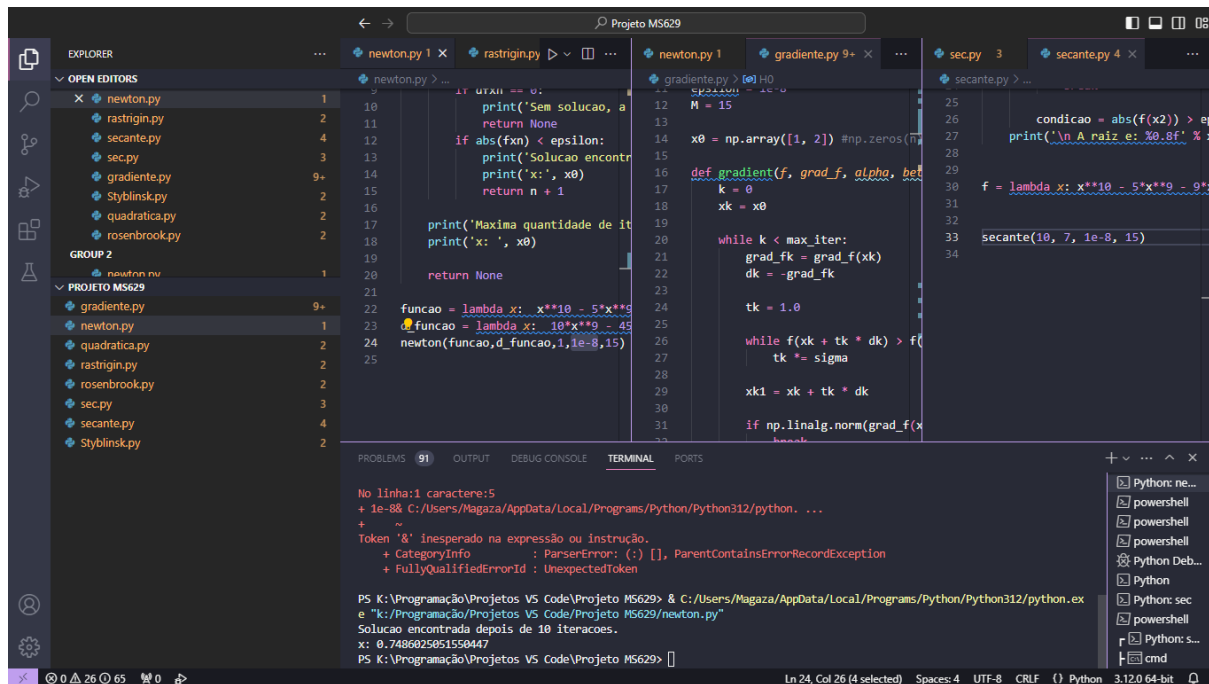


Figura 4. Comparação direta dos métodos com uma função e ponto inicial semelhante. O método de Newton converge com 4 iterações, secante com 7 e o Gradiente excede o limite de 15 iterações.

Com relação a dimensão - a quantidade de variáveis -, o método de Newton tende a se destacar se a Hessiana puder ser calculada, mesmo em casos mais complexos, mas é uma condição sem flexibilidade. Por outro lado, o método do Gradiente é mais simples, mas escala sua velocidade de convergência de forma mais severa com o aumento da dimensão sendo, no geral, o mais lento dos três. O método das Secantes é mais estável que o método de Newton para dimensões menores por exigir somente os pontos iniciais, mas com o aumento das dimensões ele se aproxima do método de Newton, até um ponto que não se difere mais nesse aspecto. O aumento de variáveis pode se tornar computacionalmente custoso, mas não foi executado nenhum caso que possa servir de exemplo.

Por fim, com relação ao ponto inicial, para todos os três métodos é crucial um valor. Todos eles são bastante sensíveis (Figura 5 e 6), porém, o método de Newton acaba por ser o mais robusto entre eles devido a sua velocidade de convergência. Ainda assim, é possível notar custos computacionais elevados ou casos sem convergência caso o ponto inicial seja mal posto, ou muito longe de um mínimo local ou global. Em particular, tem um ótimo desempenho em relação aos outros com a função de rosenbrook por causa dos vários "vales".

```

24     x1, y1 = x2, y2
25     passo = passo + 1
26
27     if passo > max:
28         print('Nao converge')
29         break
30
31     condicao = np.sqrt((f(x2, y2))**2) > epsilon
32     print('\nA raiz é: x = %0.8f, y = %0.8f' % (x2, y2))
33
34     f = lambda x, y: x**10 - 5*x**9 - 9*x**5 + 3*x**2 + x + y**5 - y**3
35
36
37     secante(22.5, 31.5, 13.3, 19.9, 1e-3, 120)
38
39

```

PROBLEMS 20 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Iteracao 15, x2 = 5.511141, y2 = 10.079265 e f(x2, y2) = 2454601.277898
Iteracao 16, x2 = 5.302807, y2 = 9.816583 e f(x2, y2) = 1056530.326005
Iteracao 17, x2 = 5.145368, y2 = 9.618072 e f(x2, y2) = 416508.874975
Iteracao 18, x2 = 5.042910, y2 = 9.488887 e f(x2, y2) = 137310.032473
Iteracao 19, x2 = 4.992522, y2 = 9.425354 e f(x2, y2) = 31302.174215
Iteracao 20, x2 = 4.977643, y2 = 9.406593 e f(x2, y2) = 3453.381438
Iteracao 21, x2 = 4.975798, y2 = 9.404267 e f(x2, y2) = 103.935142
Iteracao 22, x2 = 4.975741, y2 = 9.404195 e f(x2, y2) = 0.361900
Iteracao 23, x2 = 4.975740, y2 = 9.404195 e f(x2, y2) = 0.000038
Iteracao 24, x2 = 4.975740, y2 = 9.404195 e f(x2, y2) = 0.000000
Iteracao 25, x2 = 4.975740, y2 = 9.404195 e f(x2, y2) = 0.000000

Figura 5. Código da secante, com um ponto inicial ruim, variando somente o erro em 6 casas; a diferença de iterações é mínima.

```

21     print('Iteracao %d, x2 = %0.6f, y2 = %0.6f e f(x2, y2) = %0.6f' % (passo, x2, y2, f(x2, y2)))
22     x0, y0 = x1, y1
23     x1, y1 = x2, y2
24     passo = passo + 1
25
26
27     if passo > max:
28         print('Nao converge')
29         break
30
31     condicao = np.sqrt((f(x2, y2))**2) > epsilon
32     print('\nA raiz é: x = %0.8f, y = %0.8f' % (x2, y2))
33
34     f = lambda x, y: x**10 - 5*x**9 - 9*x**5 + 3*x**2 + x + y**5 - y**3
35
36
37     secante(4.4, 8.8, 4.1, 9.8, 1e-9, 120)
38
39

```

PROBLEMS 20 OUTPUT DEBUG CONSOLE TERMINAL PORTS

A raiz é: x = 3.68994324, y = 11.16685585
PS K:\Programação\Projetos VS Code\Projeto MS629> & C:\Users\Magaza\AppData\Local\Programs\Python\Python312\python.exe
e "k:\Programação\Projetos VS Code\Projeto MS629\sec.py"
Iteracao 1, x2 = 3.551934, y2 = 11.626887 e f(x2, y2) = 75552.723638
Iteracao 2, x2 = 3.694171, y2 = 11.152763 e f(x2, y2) = -2302.352853
Iteracao 3, x2 = 3.689965, y2 = 11.166784 e f(x2, y2) = -11.808155
Iteracao 4, x2 = 3.689943, y2 = 11.166856 e f(x2, y2) = 0.001826
Iteracao 5, x2 = 3.689943, y2 = 11.166856 e f(x2, y2) = -0.000000
Iteracao 6, x2 = 3.689943, y2 = 11.166856 e f(x2, y2) = 0.000000
A raiz é: x = 3.68994324, y = 11.16685585
PS K:\Programação\Projetos VS Code\Projeto MS629>

Figura 6. Mesmo código, mas com um ponto inicial bem melhor, com o mesmo erro 1e-9; aproximadamente 1/4 das iterações (6).

De uma forma geral, o método de Newton se mostrou superior na maioria dos casos. Para a função quadrática e Rosenbrook, é, no geral, a função que tem a melhor convergência. No caso que teve mais dificuldades, com a função Styblinsky-Tang, nenhum dos algoritmos - mesmo com o ponto inicial sendo quase a própria solução! - teve um bom desempenho com as iterações disponíveis.

The screenshot shows a VS Code editor with a file named `sec.py` open. The code defines a function `f` and a `secante` function. The terminal output shows the execution of the `secante` function with initial values `(0.4, 0.5, 0.1, 0.1, 1e-8, 10000000)`. The output displays 12 iterations of the Secant method, showing the values of `x2`, `y2`, and `f(x2, y2)` for each iteration.

```

25     passo = passo + 1
26
27     if passo > max:
28         print('Nao converge')
29         break
30
31     condicao = np.sqrt((f(x2, y2))**2) > epsilon
32     print('\nA raiz é: x = %0.8f, y = %0.8f' % (x2, y2))
33
34     f = lambda x, y: x**2 + 2*y**2
35
36
37
38     secante(0.4, 0.5, 0.1, 0.1, 1e-8, 10000000)
39

```

```

Iteracao 3318513, x2 = 0.018365, y2 = -0.008846 e f(x2, y2) = 0.000494
Iteracao 3318514, x2 = 0.017280, y2 = -0.010293 e f(x2, y2) = 0.000510
Iteracao 3318515, x2 = 0.050467, y2 = 0.033948 e f(x2, y2) = 0.004852
Iteracao 3318516, x2 = 0.013378, y2 = -0.015495 e f(x2, y2) = 0.000659
Iteracao 3318517, x2 = 0.007546, y2 = -0.023269 e f(x2, y2) = 0.001140
Iteracao 3318518, x2 = 0.021375, y2 = -0.004835 e f(x2, y2) = 0.000504
Iteracao 3318519, x2 = 0.032322, y2 = 0.009758 e f(x2, y2) = 0.001235
Iteracao 3318520, x2 = 0.013838, y2 = -0.014881 e f(x2, y2) = 0.000634
Iteracao 3318521, x2 = -0.005681, y2 = -0.040902 e f(x2, y2) = 0.003378
Iteracao 3318522, x2 = 0.018351, y2 = -0.008865 e f(x2, y2) = 0.000494
Iteracao 3318523, x2 = 0.022467, y2 = -0.003379 e f(x2, y2) = 0.000528
Iteracao 3318524, x2 = -0.042069, y2 = -0.089409 e f(x2, y2) = 0.017758

```

Figura 8. Método da Secante executando a função quadrática com duas variáveis num contexto de 10 milhões de iterações; método de Newton com três variáveis convergiu em somente 15.

The screenshot shows a VS Code editor with a file named `newt.py` open. The code defines a function `f` and a `newton` function. The terminal output shows the execution of the `newton` function with initial values `(0.4, 0.5, 0.1, 0.1, 1e-8, 10000000)`. The output displays 12 iterations of the Newton method, showing the values of `x2`, `y2`, and `f(x2, y2)` for each iteration. The output also shows the final solution found after 279 iterations.

```

fxn = f(x0, y0, z0)
^^
UnboundLocalError: cannot access local variable 'z0' where it
PS K:\Programação\Projetos VS Code\Projeto MS629> & C:/Users/M/
e "k:/Programação/Projetos VS Code/Projeto MS629/newt.py"
Maxima quantidade de iteracoes atingida. Nenhuma solucao encon
x: 1.2139672700492645 1.2139672700492645
PS K:\Programação\Projetos VS Code\Projeto MS629> & C:/Users/M/
e "k:/Programação/Projetos VS Code/Projeto MS629/newt.py"
Solucao encontrada depois de 279 iteracoes.
x: 1.0000947980261652 1.0000947980261652
PS K:\Programação\Projetos VS Code\Projeto MS629>

```

```

Iteracao 9993, x2 = 0.832190, y2 = 0.842924 e f(x2, y2) = 0.031393
Iteracao 9994, x2 = 0.850776, y2 = 0.841731 e f(x2, y2) = 0.024611
Iteracao 9995, x2 = 0.918223, y2 = 0.837404 e f(x2, y2) = 0.208005
Iteracao 9996, x2 = 0.841725, y2 = 0.842312 e f(x2, y2) = 0.025061
Iteracao 9997, x2 = 0.831246, y2 = 0.842984 e f(x2, y2) = 0.032340
Iteracao 9998, x2 = 0.877801, y2 = 0.839998 e f(x2, y2) = 0.057102
Iteracao 9999, x2 = 0.770442, y2 = 0.846885 e f(x2, y2) = 0.205550
Iteracao 10000, x2 = 0.919097, y2 = 0.837348 e f(x2, y2) = 0.212718
Nao converge
A raiz é: x = 0.91909698, y = 0.83734835
PS K:\Programação\Projetos VS Code\Projeto MS629>

```

Figura 9. Método de Newton e da Secante executando a função de Rosenbrock com o método de Newton tendo um desempenho bem melhor comparativamente.

```

25     passo = passo + 1
26
27     if passo > max:
28         print('Nao converge')
29         break
30
31     condicao = np.sqrt((f(x2, y2))**2) > epsilon
32     print('\nA raiz é: x = %0.8f, y = %0.8f' % (x2, y2))
33
34     f = lambda x, y: x**2 + y**2 - 10*math.cos(2*3.14*x) - 10*math.cos(2*3.14*y)
35
36
37
38     secante(0, 0.5, 0.3, 0.2, 1e-8, 10000)
39

```

PROBLEMS 31 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

Iteracao 9993, x2 = 0.924479, y2 = -0.424479 e f(x2, y2) = 1.042150
Iteracao 9994, x2 = 0.294820, y2 = 0.205180 e f(x2, y2) = 0.113718
Iteracao 9995, x2 = 0.217698, y2 = 0.282302 e f(x2, y2) = 0.111487
Iteracao 9996, x2 = -3.636235, y2 = 4.136235 e f(x2, y2) = 30.318753
Iteracao 9997, x2 = 0.231921, y2 = 0.268079 e f(x2, y2) = 0.109830
Iteracao 9998, x2 = 0.245985, y2 = 0.254015 e f(x2, y2) = 0.109111
Iteracao 9999, x2 = 2.380438, y2 = -1.880438 e f(x2, y2) = 9.191581
Iteracao 10000, x2 = 0.220343, y2 = 0.279657 e f(x2, y2) = 0.111108
Nao converge

A raiz é: x = 0.22034280, y = 0.27965720
PS K:\Programação\Projetos VS Code\Projeto MS629>

```

Figura 10. Método da Secante não convergindo mesmo com 10 mil interações.

```

38     secante(-3, -2.9, -2.9, -2.7, 1e-8, 10000)
39
40

```

PROBLEMS 33 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

Iteracao 95, x2 = 6.861771, y2 = 16.823542 e f(x2, y2) = -2067.368929
Iteracao 96, x2 = -31.099828, y2 = -59.099657 e f(x2, y2) = 11992710.00090
Iteracao 97, x2 = 6.855228, y2 = 16.810457 e f(x2, y2) = -2189.208448
Iteracao 98, x2 = 6.848301, y2 = 16.796602 e f(x2, y2) = -2317.665275
Iteracao 99, x2 = 6.973285, y2 = 17.046570 e f(x2, y2) = 86.276504
Iteracao 100, x2 = 6.968799, y2 = 17.037598 e f(x2, y2) = -3.195408
Iteracao 101, x2 = 6.968959, y2 = 17.037919 e f(x2, y2) = -0.004144
Iteracao 102, x2 = 6.968960, y2 = 17.037919 e f(x2, y2) = 0.000000
Iteracao 103, x2 = 6.968960, y2 = 17.037919 e f(x2, y2) = -0.000000

A raiz é: x = 6.96895961, y = 17.03791922
PS K:\Programação\Projetos VS Code\Projeto MS629>

```

```

28     newton(funcao, d_funcao, -2.9, -2.8, 1e-8, 10000)
29
30

```

PROBLEMS 33 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

e "k:/Programação/Projetos VS Code/Projeto MS629/newt.py"
Solucao encontrada depois de 12 iteracoes.
x: -4.1020767071492614e-29 -4.1020767071492614e-29
PS K:\Programação\Projetos VS Code\Projeto MS629> & C:/Users/Magaza/AppData/Local/Progr
e "k:/Programação/Projetos VS Code/Projeto MS629/newt.py"
Solucao encontrada depois de 12 iteracoes.
x: -4.1020767071492614e-29 -4.1020767071492614e-29
PS K:\Programação\Projetos VS Code\Projeto MS629> & C:/Users/Magaza/AppData/Local/Progr
e "k:/Programação/Projetos VS Code/Projeto MS629/newt.py"
Solucao encontrada depois de 724 iteracoes.
x: 15.939834410726068 16.039834410726066
PS K:\Programação\Projetos VS Code\Projeto MS629>

```

Figura 11. Nem a Secante e nem o método de Newton tiveram bons resultados com a função Styblinsky-Tang.

Algumas observações sobre o manuseio geral das funções do trabalho; os algoritmos foram implementados inicialmente com uma variável, sendo posteriormente testado com mais variáveis. Ambos algoritmos tanto da Secante quanto de Newton são executados com a função objetivo sendo inputada pelo usuário manualmente, o que inclui o manejo das variáveis.

Com relação as funções de teste - quadrática, rosenbrook, rastrigin e Styblinsky -, elas foram implementadas de modo a fornecer as funções que são imputadas nos algoritmos. Por exemplo, a função quadrática, quando implementada, fornece a função com n variáveis e sua derivada para ser usada, caso necessária. Também foi utilizada, em alguns casos, uma função auxiliar para calcular a hessiana e o gradiente.

Todas essas funções estão linkadas junto com a entrega do pdf do relatório.